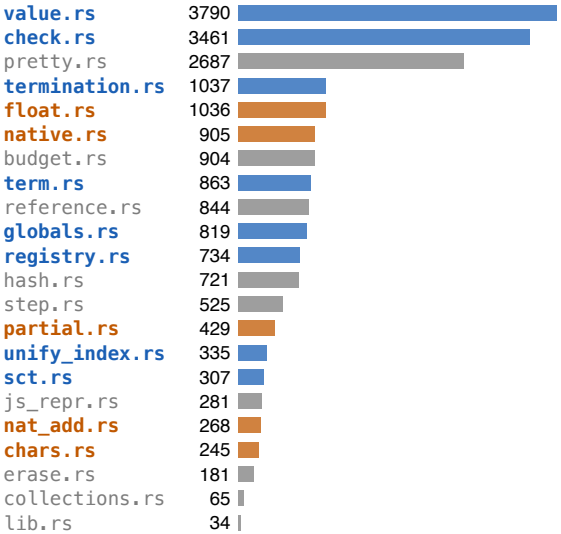
`stitchwort-core`: the core language and the trusted kernel

What it does

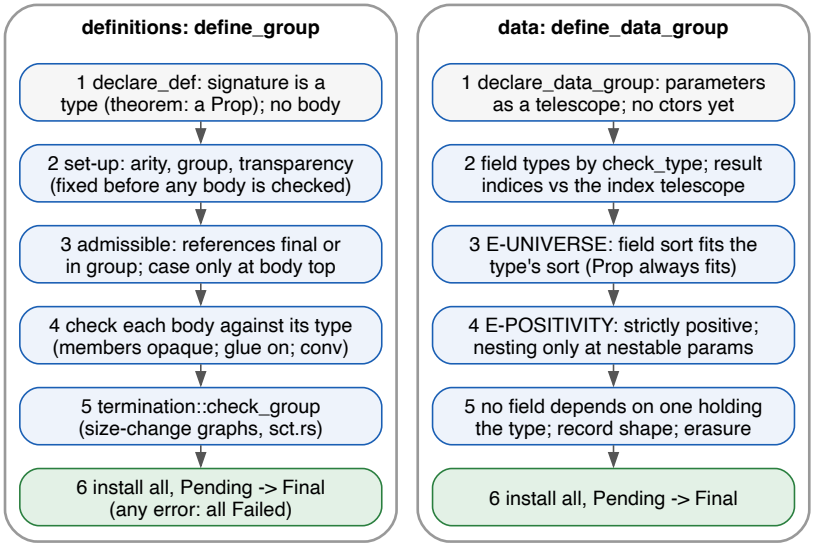
The crate defines the **core language** every definition is translated into, and the **kernel** that re-checks the elaborator's output from scratch (`crates/elab` is not trusted; DESIGN.md §11). A bug in the kernel can make a false proof check; a bug anywhere else in the workspace cannot. It is also where the rest of the toolchain meets: `elab`, `driver`, `rt`, `js`, `native`, `standalone`, `web` and `cli` all depend on it.

The crate holds more than the kernel. **Blue** = kernel, **orange** = trusted computation rules (native shortcuts a bug in which could make `2 + 2 = 5` check), **grey** = not trusted. Lines are `src/` only (20471 in all; kernel 11346, rules 2883, support 6242).



- **Kernel:** `term` (core terms), `value` (NbE: eval, apply, quote, conversion), `check` (infer/check, admission of defs and data), `globals` (declarations), `termination` + `sct` (size change), `unify_index` (indices for `case`), `registry` (prelude built-ins by identity).
- **Rules:** `native` (`Nat` as a bignum, `Bool`, `Decidable`), `nat_add` (+ also computes on its second argument), `float` (IEEE doubles), `chars` (compact `Text`), `partial` (`Delay` and its constants).
- **Support:** `pretty` (terms and values to surface-like text, for messages and the REPL), `reference` (straightforward recursive evaluator, feature `reference`/tests, for differential tests of `value`), `erase` (skip proofs and types while running), `hash` (content hashes), `budget`, `step` (run mode of `Delay`), `js_repr` (JS tag keys, never read by the kernel), `collections` (deterministic hash maps).
- **For the elaborator:** `Term::Meta`, metavariables in `Globals::metas` with solutions, a trail and `meta_epoch`, `Value::Neutral(Head::Meta, ..)`, and `Relation::Unify` in conversion. The kernel only accepts complete terms.

How a declaration is admitted



- **Order.** Types first (signatures, field types), then bodies, then termination, then install. Definitions and data are admitted in **groups** (mutual recursion); a group of one is the usual case. Either every member becomes `Final` or every one is `Failed`.
- **Check cache.** `admit_trusted_*` re-installs declarations accepted in an earlier run without checking (trust boundary: the cache files). `--no-cache` never uses them.
- **Termination.** Each call between members becomes a size-change `Graph` (<, ≤ or unknown per argument). Accepted if every idempotent self-graph in the composition closure has a strict diagonal arc; `MAX_GRAPHS` bounds the closure.

Evaluation, quotation, conversion (NbE, `value.rs`)

- `eval(g, env, term)` runs an explicit machine: `Control` (`Eval`, `Apply`, `Unfold`, `Tree`, `Force`, `Return`) over a heap `Vec<Frame>`, so deep programs don't use the Rust stack. `quote` (`QuoteTask`) and `conv` (`ConvTask`) have their own work stacks, so they are stack-safe too.
- **Unfolding policy:** a definition stays folded (`Neutral(Head::Def, spine)`) until it has all its parameters and its case tree can step; dictionaries (`fold`) and theorems stay folded until something takes them apart.
- **Glued calls:** with `Globals::glue` on, a saturated call is `Value::Call`; its unfolding is computed on first `force`, then shared. The kernel glues only while checking a body.
- **Conversion** `conv(g, level, a, b)`: structural with eta for functions; folded calls of one definition are compared by arguments first, then unfolded; two non-recursive definitions stuck on `case` are compared by case trees; numbers compare as `k + x`. Memoised per comparison (`CallMemo`, `TreeMemo`).
- **Readback:** variables are levels in values and indices in terms; `quote` converts, and `quote_folded` keeps calls as calls.

Budgets and resource exhaustion (`budget.rs`)

A `Budget` lives in `Globals`. `operation(Limits)` starts one top-level operation; `Limits { work, stack, size, frames }`. Each machine transition (and each quote and conversion step) calls `Budget::step`; stack position, frame count and output size are checked too, plus an optional deadline and a cancel flag (language servers). Running out is a separate `ResourceError` (`Work`, `Depth`, `Nesting(pass)`, `Size`, `Deadline`, `Cancelled`), propagated by every evaluation, quotation and conversion as `Result<T>`; the kernel reports it as E-RESOURCE. It is inconclusive, not a type error: the group fails, and later checks fail until the next operation.

Central data types

`Term` (`term.rs`; de Bruijn indices; `Rc` children)

- vars and binders: `Var(usize)`, `Def(DefId)`, `Pi(Name, Icit, dom, cod)`, `Lam(Name, Icit, body)`, `App(f, a, Icit)`, `Ann(t, ty)`, `Let(Name, A, v, b, erased)`
- data: `Data(DataId, params)`, `Ctor(DataId, ctor, fields)`, `Case(var, DataId, Rc<[Branch]>)` and `NumCase(var, Rc<[NumArm]>, default)` (only at the top of a body, forming a case tree); `Branch { names, body: Option<..> }`, none = impossible
- literals and universes: `Nat(BigUint)`, `Chars(Chars)`, `Sort(Sort)`; `Sort = Prop | Type(u32)` up to `MAX_LEVEL` (65535)
- equality, quotients: `Eq(Option<ty>, a, b)`, `Refl`, `J(motive, base, proof)`, `Quot(QuotForm, args, erased)`; `QuotForm = Type | Mk | Lift | Ind | Sound`
- bookkeeping: `Loc(Span, t)` (errors only), `Meta(usize)` (elaborator only)

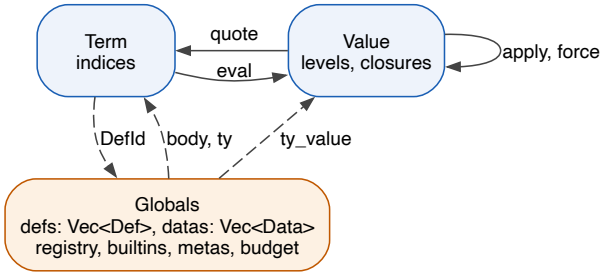
`Value` (`value.rs`; de Bruijn levels)

- `Sort`, `Pi(Name, Icit, Rc<Value>, Closure)`, `Lam(Name, Icit, Closure)`, `Data`, `Ctor(DataId, usize, Values)`, `Nat(BigUint)`, `NatAdd(k, Rc<Value>)` (`k + x`), `Chars`, `Eq`, `Refl`
- `Neutral(Head, Spine)`: stuck. `Head = Var(level, Apart) | Def(DefId) | Meta(usize) | Quot(QuotForm)`; `Spine` is a shared list of `Elim = App(Value, Icit) | J { motive, base }`
- `Call(DefId, Spine, Rc<Unfolding>)`: glued call with a lazily shared unfolding; `Resumption`: runner only
- `Closure { env: Values, body: Rc<Term> }`; `Values` is a shared `Rc<[Value]>`

`Globals` (`globals.rs`): `defs: Vec<Def>`, `datas: Vec<Data>`, `builtins: Builtins`, `registry: Registry`, `metas`, `hashes`, `budget`, `erasure`, `glue`, `meta_epoch`, `transparent`.

- `DefId(usize)` and `DataId(usize)` are indices into `defs` and `datas`. There is no `GlobalId`; `HashKey = Def(DefId) | Data(DataId)`.
- `Def`: `name`, `path`, `ty: Term`, `ty_value: Value`, `body: Option<Rc<Term>>`, `state`, `arity`, `group`, flags `theorem` `instance` `fold` `opaque` `recursive` `export`, `unfolding`, `run_body`, `validators`, `span`
- `Data`: `params`, `indices`, `sort`, `ctors: Vec<Ctor>`, `record`, `group`, `nestable`, `state`; `Ctor`: `name`, `fields`, `field_sorts`, `field_erased`, `result` (indices), `implicit`
- `State = Pending | Final | Failed`. `Registry` maps `Builtin` (prelude names the language means by identity) to ids. `ContentHash([u8; 32])` per `HashKey`.

`Budget`, `Limits`, `ResourceError`, `Ctx` (checking context: `env`, `types`, `names`, `span`): see left.



Where to start reading. `check.rs`: `define_group` (via `install_group` and `admit_group`), `declare_def`, `define_data_group`, then `infer` and `check`. `value.rs`: `eval`, `quote`, `conv`. `term.rs`: `Term`. `globals.rs`: `Def`, `Data`, `Globals`. The rules are in `docs/core.md`.